

Grant Miller

EE 446

25 May 2026

Lab 8 Report

Part I:

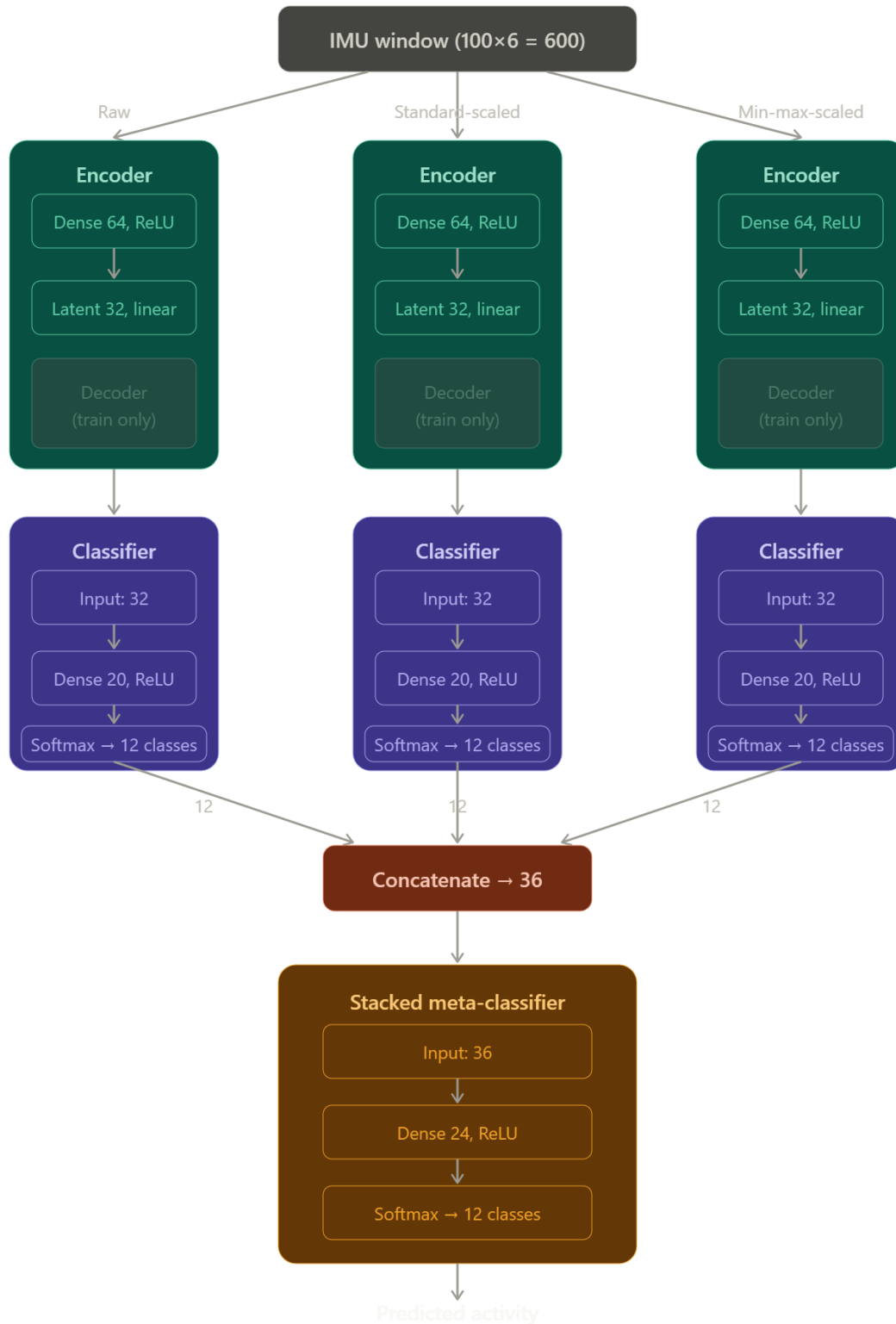
Edge Impulse Link: <https://studio.edgeimpulse.com/public/1008753/live>

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 34 ms, inference 570 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  Idle: 0.012869
  Lift: 0.987131
```

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 34 ms, inference 570 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  Idle: 0.994782
  Lift: 0.005218
```

Part II: Tiny Ensemble Learning

Question 1: Model architecture and ensemble flow



Question 2: Pruning and QAT methodology

This notebook uses a few different techniques to compress the models for TinyML deployment.

- This notebook uses `prune_low_magnitude` which zeros out the weights with the smallest magnitudes. The sparsity schedule being used here is `PolynomialDecay` that ramps sparsity from 10% all the way to 80% over 500 training steps, trained over 5 epochs. So, the model starts with most of the weights present and slowly removes more.
- Pruning wrappers are stripped before QAT because the `quantize_model()` function needs to wrap each layer in its own `QuantizedWrapper` and it isn't possible to nest two different wrappers together. So, stripping before produces a plain model that can be wrapped again.
- QAT is applied in the following manner: after stripping, the `quantize_model()` function wraps the stripped model with fake-quantization nodes meant to simulate int8 rounding during the forward passes. The QAT model is then trained for 5 more epochs so the weights learn to compensate for quantization errors.
- The mask-enforcement step is needed because once the pruning wrappers are stripped and QAT training starts, it's still possible for the optimizer to revive and use weights that were previously zeroed. The `MaskEnforcerCallback` solves this by recording which weights were zero after pruning then re-zeroing them after every training batch and epoch during QAT.
- The final model is converted into a int8 TFLite model starting with the converter using `tf.lite.Optimize.DEFAULT` with `target_spec.supported_ops = [TFLITE_BUILTINS_INT8]` and setting both input/output types to int8. Full integer quantization requires knowing the range of activations at each layer to calculate the proper scale/zero-point parameters. Since activation ranges depend on the actual data distribution, the converter must run the model on 200 representative samples to calibrate these ranges.
- The Arduino Nano 33 BLE has a very limited 256 KB of SRAM and only 1 MB flash memory. Pruning and int8 quantization both significantly reduce the model size by cutting out many unnecessary parts of the model like zero weights and 32 bit floats.

Question 3: Arduino deployment and real-world behavior

```
Tiny Ensemble Learning inference sketch
IMU initialized. Collecting 100-sample windows at 50 Hz.
Waiting for accelerometer and gyroscope data...
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.4570, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.3281, 0.2148, 0.0000
Standard-scaled model scores: 0.9727, 0.0078, 0.0078, 0.0000, 0.0078, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039
Min-max-scaled model scores: 0.9297, 0.0078, 0.0508, 0.0000, 0.0000, 0.0117, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.7930, 0.0000, 0.1016, 0.0000, 0.0000, 0.0625, 0.0078, 0.0000, 0.0000, 0.0234, 0.0039, 0.0039
-----
Final prediction: Standing still
Class index: 0
Confidence score: 0.7930
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.4570, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.3281, 0.2148, 0.0000
Standard-scaled model scores: 0.9727, 0.0117, 0.0078, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039
Min-max-scaled model scores: 0.9297, 0.0078, 0.0508, 0.0000, 0.0000, 0.0117, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.8047, 0.0000, 0.0977, 0.0000, 0.0000, 0.0586, 0.0078, 0.0000, 0.0000, 0.0195, 0.0039, 0.0039
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.4570, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.3281, 0.2148, 0.0000
Standard-scaled model scores: 0.9727, 0.0117, 0.0078, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039
Min-max-scaled model scores: 0.9297, 0.0078, 0.0508, 0.0000, 0.0000, 0.0117, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.8047, 0.0000, 0.0977, 0.0000, 0.0000, 0.0586, 0.0078, 0.0000, 0.0000, 0.0195, 0.0039, 0.0039
-----
Final prediction: Standing still
Class index: 0
Confidence score: 0.8047
-----
```

This output is acceptable for the resting board condition as it predicted standing still with fairly high confidence many times in a row when the board was at rest. There are however noticeable issues as in certain cases, three branch models disagree significantly. The raw branch thinks the board is "Running" (class 10, ~80% confidence), which is clearly wrong. The standard-scaled branch leans toward "Standing still" (~61%) but also gives meaningful weight to "Climbing stairs" (~16%) and "Lying down" (~14%). Only the min-max branch is strongly confident in "Standing still" (~97%). The confidence of ~75% is also only moderate rather than high for a completely scenario that should be clear cut. Ideally you'd expect 90%+ for a motionless board.

Some practical changes that could improve deployment reliability include collecting calibration data directly from the Arduino in each target orientation and either fine-tune the model on that data or add it to the training set. This would close the domain gap between the mHealth body-sensor data and the board sitting on a desk. Additionally, applying a simple post-processing filter such as majority voting over the last 3 to 5 consecutive predictions to mitigate any transition period misclassifications would improve stability in deployment without changing the model itself.

Question 4: Deployment Behavior on the Arduino

Yes, there were some unexpected predictions where I would be moving the board in a constant up and down motion to try and simulate a walking movement, and the output would be the occasional walking classification but also a bunch of random other classes.

This might happen because there is a mismatch between the mHealth training data and the live Arduino sensor data. The mHealth dataset was recorded from IMU sensors worn on specific body locations. The gravity vector and motion patterns at those positions are very different from what the Arduino reads when hand-held or lying on a table. Another possible reason is feature scaling calibration as the live data can possibly produce values outside the expected range. Lastly, motion pattern mismatch everyone moves differently and without actually wearing the device, it is hard to tell if the movement is close to what the model was trained on.